Module 5

Regression Management & Advanced UVM Orchestration (SV/UVM)

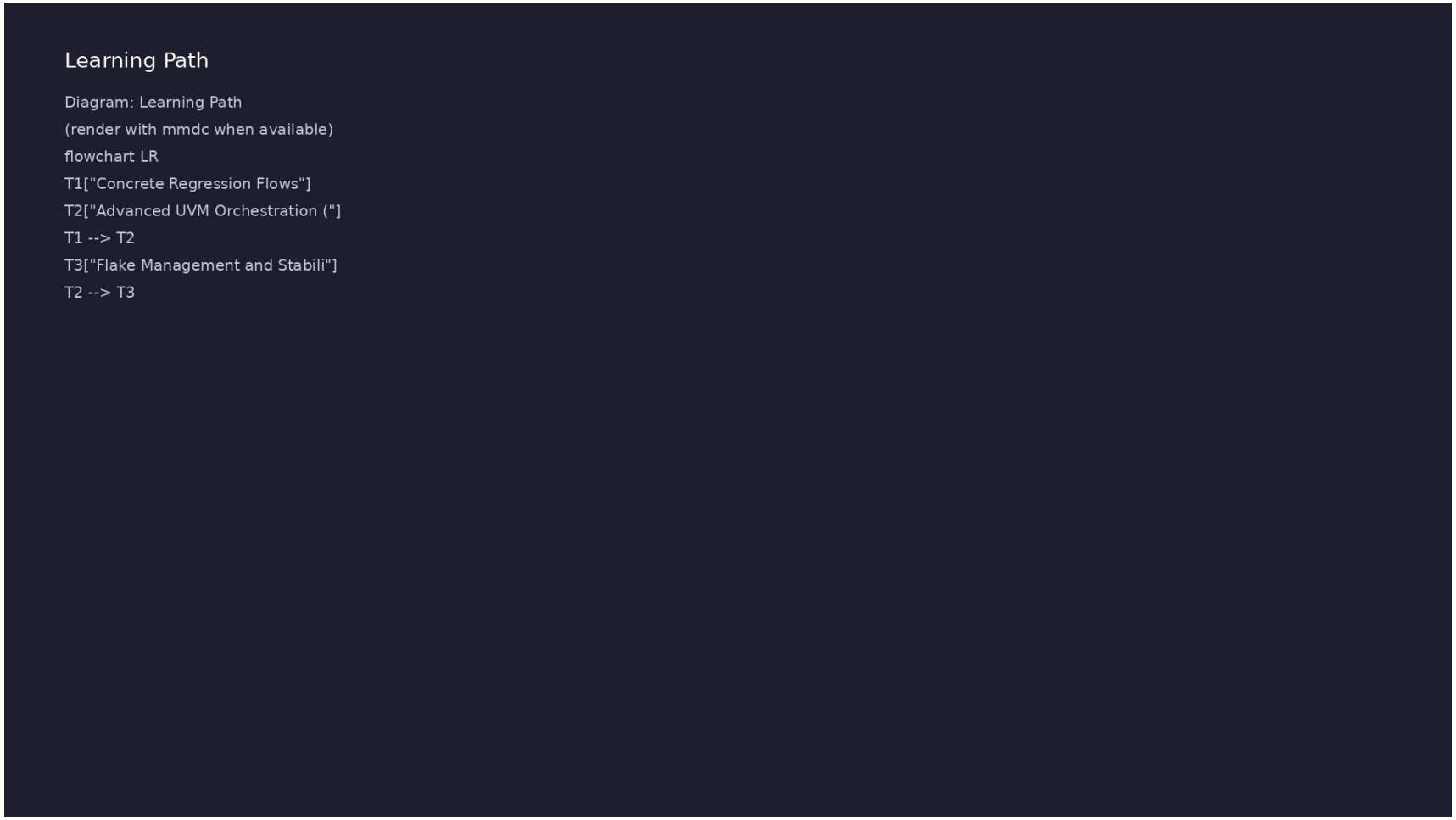
Learning objectives

- Turn your test, coverage, and environment designs into a **robust regression system** that scales:…

Prerequisites

- See module README

Learning path



Turn your test, coverage, and environment designs into a **robust regression system** that scales: advanced sequence orchestration (includi…

Overview

- Modules 1–4 established **what to test**, **how to measure**, and **how to build a solid UVM enviro…

How to learn this module

Suggested learning path (1/2)

- Re-read `module2/REGRESSION_PLAN.md` and `module4/ENV_DESIGN.md`.
- Scaffold Module 5 workspace: `./scripts/module5.sh --scaffold`
- Document regression ops in `module5/REGRESSION_OPS.md` and advanced UVM in `ADVANCED_UVM_PLAN.md`.
- Plan flake handling in `DEBUG_FLAKE_PLAN.md`.
- Implement virtual sequences, callbacks, or config patterns in `common_dut/tb/`.

Suggested learning path (2/2)

- Validate: `./scripts/module5.sh --check`

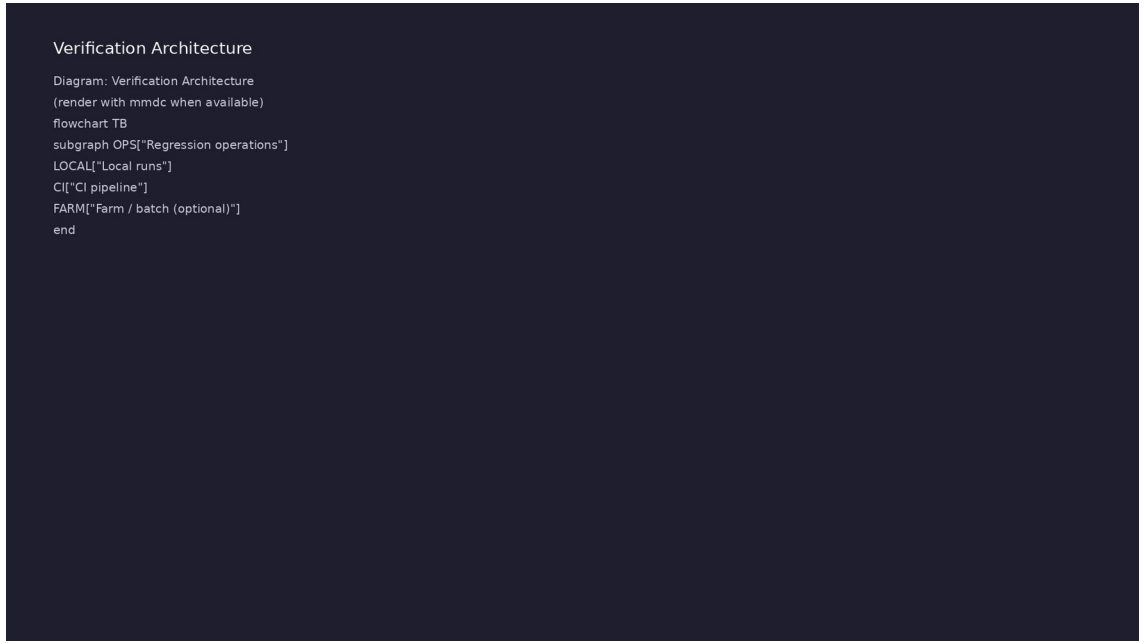
Design architecture

1. Regression system architecture

- Local quick loops, CI gated merges, optional farm for long suites (`REGRESSION_OPS.md`).
- Regression launcher selects tier → test list → seeds → coverage merge → report artifacts.

2. Advanced UVM orchestration

- Virtual sequences coordinate multiple agents (e.g., concurrent source/sink traffic).
- Callbacks and config DB patterns for mode switches without rewriting tests.
- Documented in `ADVANCED_UVM_PLAN.md`; implemented under `common_dut/tb/`.



3. Regression launch and triage pipeline

- Launch: tier script selects test list, seeds, plusargs, and parallelism level.
- Execute: each test produces log, optional waveform, and coverage database.
- Collect: merge coverage, aggregate pass/fail, and flag timeouts or assertion failures.
- Triage: reproduce with saved seed; use `DEBUG_FLAKE_PLAN.md` for intermittent failures.
- Report: publish tier summary for sign-off review and nightly dashboard consumption.

Monitor — analysis port

```
class stream_monitor extends uvm_component;

  `uvm_component_utils(stream_monitor)

  virtual stream_if.slave vif;
  uvm_analysis_port #(stream_item) ap;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    ap = new("ap", this);
  endfunction

  task run_phase(uvm_phase phase);
    stream_item tr;

    if (vif == null) begin
# ...
```

stream_test — run_phase

```
class stream_test extends uvm_test;

  `uvm_component_utils(stream_test)

  stream_env env;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = stream_env::type_id::create("env", this);
  endfunction

  task run_phase(uvm_phase phase);
# ...
```

Key files to study

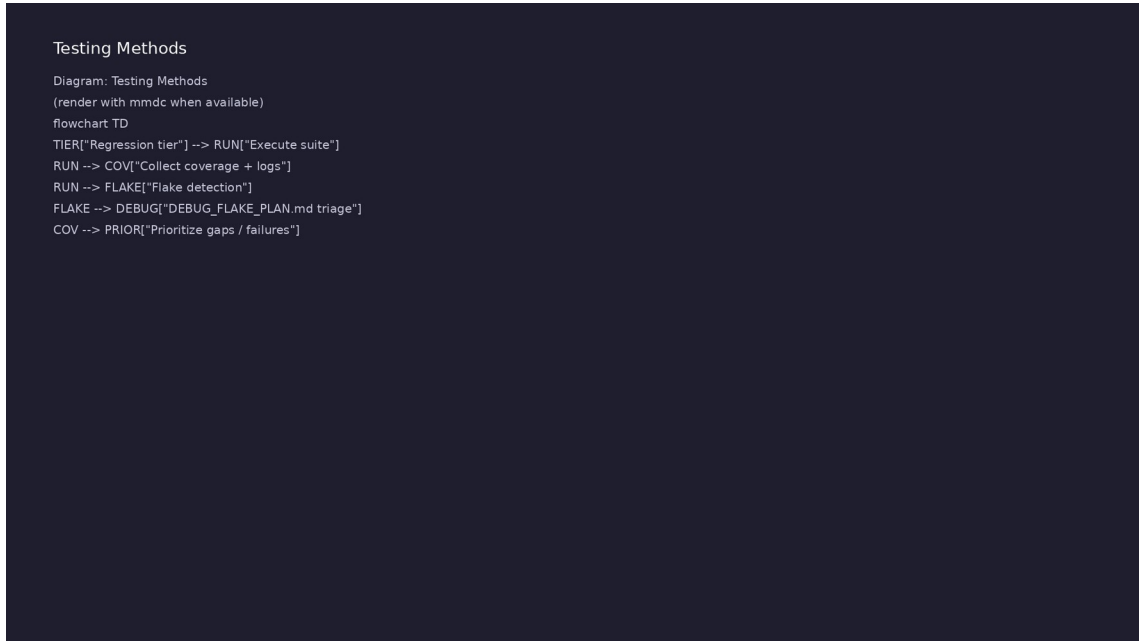
Open these in the repo

- `module5/REGRESSION_OPS.md` — tiers, launchers, CI/farm integration
- `module5/ADVANCED_UVM_PLAN.md` — virtual sequences, callbacks, config DB usage
- `module5/DEBUG_FLAKE_PLAN.md` — flake detection, quarantine, debug workflow
- `module4/tb/stream_pkg.sv` — monitor, test, and sequence patterns to extend
- `scripts/module5.sh` — regression and advanced-UVM doc checks

Verification & testing methods

1. Operational regression testing

- Every tier has target runtime, pass criteria, and artifact retention (logs, waves, coverage DB).
- Failures triaged with reproducible seeds and minimized waveforms.



2. Flake and performance management

- `DEBUG_FLAKE_PLAN.md` — detect flaky tests, quarantine policy, timeout budgets.
- Performance tuning: parallel jobs, test ordering, and dropping redundant long tests.

3. Data-driven refinement

- Use regression + coverage history to prioritize new tests and retire low-value ones.
- `./scripts/module5.sh --check` validates ops and advanced-UVM planning docs.

Module 5 — regression validation

```
#!/usr/bin/env bash

# Module 5: Regression & Advanced UVM Orchestrator
# This script summarizes and checks the artifacts for Module 5.
# Usage: ./scripts/module5.sh [OPTIONS]

set -euo pipefail

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
```

Syllabus topics

1. Concrete Regression Flows (1/2)

- Local vs CI vs Farm
- Developer “inner loop” (fast, focused subsets).
- CI per-commit/per-PR runs (sanity + selected CORE tests).
- Nightly/weekly farm runs (CORE/STRESS/FULL tiers).
- Command-Line Interfaces
- Standardizing make/pytest/regression wrapper interfaces.

1. Concrete Regression Flows (2/2)

- Passing seeds, tiers, and configuration via plusargs/env/CLI options.
- Job Definitions
- Defining named regression jobs (e.g., `sanity`, `core_nightly`, `stress_weekly`).
- Mapping jobs to tiers, test lists, and resource limits.

2. Advanced UVM Orchestration (Virtual Sequences & Multi-Agent) (1/2)

- Virtual Sequencers & Virtual Sequences
- Coordinating multiple agents (e.g., data + control + sideband).
- Layered sequences that correspond to system-level scenarios.
- Mapping to Test Plan
- Mapping high-level test intents to virtual sequences.
- Using virtual sequences to implement complex scenarios (reset-in-flight, multi-channel flows).

2. Advanced UVM Orchestration (Virtual Sequences & Multi-Agent) (2/2)

- Configuration & Callbacks
- Using complex configuration objects to parameterize regressions.
- Using callbacks to inject debug behavior or temporary instrumentation.

3. Flake Management and Stability (1/2)

- Flaky Test Identification
- Signals of flakiness (non-deterministic failures, timeouts).
- Logging and metadata needed to diagnose flakiness (test ID, seed, environment).
- Policies
- Criteria for marking a test as flaky.
- Temporary quarantine vs immediate fix.

3. Flake Management and Stability (2/2)

- Automatic rerun strategies (e.g., N-of-M reruns to confirm flakiness).
- Designing Flake-Resistant Tests
- Avoiding hidden dependencies on timing or environment.
- Ensuring proper resets and deterministic seeding.

4. Debug & Observability Strategy (1/2)

- Logging
- Standardized log formats (prefixes, IDs, levels).
- Per-test log collection and retention policies.
- Waveforms and Traces
- When to dump waves (always for failures, selectively for passes).
- How to configure trace windows (e.g., last N microseconds before failure).

4. Debug & Observability Strategy (2/2)

- Failure Triage Workflow
- Quick checks (assertion vs scoreboard vs timeout).
- Reproduction steps (same test, same seed, smaller subset).

5. Performance and Scalability (1/2)

- Bottleneck Identification
- Longest-running tests and why.
- Hot spots in sequences, drivers, or reference models.
- Optimization Strategies
- Reducing redundant work (e.g., fewer warm-up cycles, targeted stress).
- Using configuration to scale test depth per tier.

5. Performance and Scalability (2/2)

- Regression Capacity Planning
- Estimating how many tests can run per hour/day.
- Planning parallelism and job sharding.

6. Coverage-Driven Regression Refinement

- Using Coverage to Shape Regressions
- Prioritizing tests that contribute most to coverage gaps.
- Designing “closure suites” for specific coverage areas.
- Bug-Driven Refinement
- Using bug history to prioritize tests and sequences.
- Ensuring that fixed bugs get regression tests and coverage.

Command reference highlights

Scaffold and validate Module 5

- ./scripts/module5.sh --scaffold

Inspect test run_phase and monitor hooks

- grep -n "class stream_test" module4/tb/stream_pkg.sv

Hands-on examples

Module 5 self-check

```
$ ./scripts/module5.sh --help
```

Terminal: module5_check

```
[0;36m==========================================[0m
[0;36mModule 5: Regression Management & Advanced UVM Orchestration[0m
[0;36m==========================================[0m

Usage: ./scripts/module5.sh [OPTIONS]

Module 5: Regression Management & Advanced UVM Orchestration (SV/UVM)
This script summarizes and checks the planning artifacts for Module 5.

OPTIONS:
    --regression-status    Show status of regression_ops.md only
    --uvm-status           Show status of advanced_uvm_plan.md only
    --debug-status         Show status of debug_flake_plan.md only
    --checklist-status     Show status of checklist_module5.md only
    --summary              Show all statuses (default)
    --check                Media/CI self-check (structure only)
    --demo                 Print example commands from EXAMPLES.md
    --scaffold             Copy templates/*.md into module5/ if missing
    --help, -h             Show this help message

EXAMPLES:
    # Full summary
    ./scripts/module5.sh

    # Focus only on regression operations
    ./scripts/module5.sh --regression-status
```

Exercise scaffold

```
./scripts/module5.sh --scaffold
```

Demo: Regression operations template

```
$ head -30 module5/templates/REGRESSION_OPS.md
```

Terminal: ex01_regression_ops

```
# Regression Operations (Module 5)

> **Purpose**: Define concrete regression flows, commands, and CI/farm integration.
> **Module**: 5 – Regression Management & Advanced UVM Orchestration (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module5/templates/` if nee

## 1. Context

<!-- TODO: DUT/Project, primary simulator(s), earlier plans (tiers, coverage, env). -->

## 2. Regression Jobs

<!-- TODO: Table of Job Name, Description, Tier(s), Approx Runtime, Intended Use. -->

## 3. Command-Line Interfaces

### 3.1 Local (Developer) Usage

<!-- TODO: Example commands (run_regression.sh or equivalent), options, environment variables. -

### 3.2 CI / Farm Usage

<!-- TODO: CI/farm integration, job definitions (YAML etc.), parameters. -->

## 4. Test Selection Rules

<!-- TODO: How each job chooses tests (by tier, ID, tag). Link to module2/TEST_PLAN.md. -->

## 5. Seeds and Randomization at Regression Level
```

Demo: Debug and flake plan

```
$ head -25 module5/templates/DEBUG_FLAKE_PLAN.md
```

Demo: Advanced UVM plan

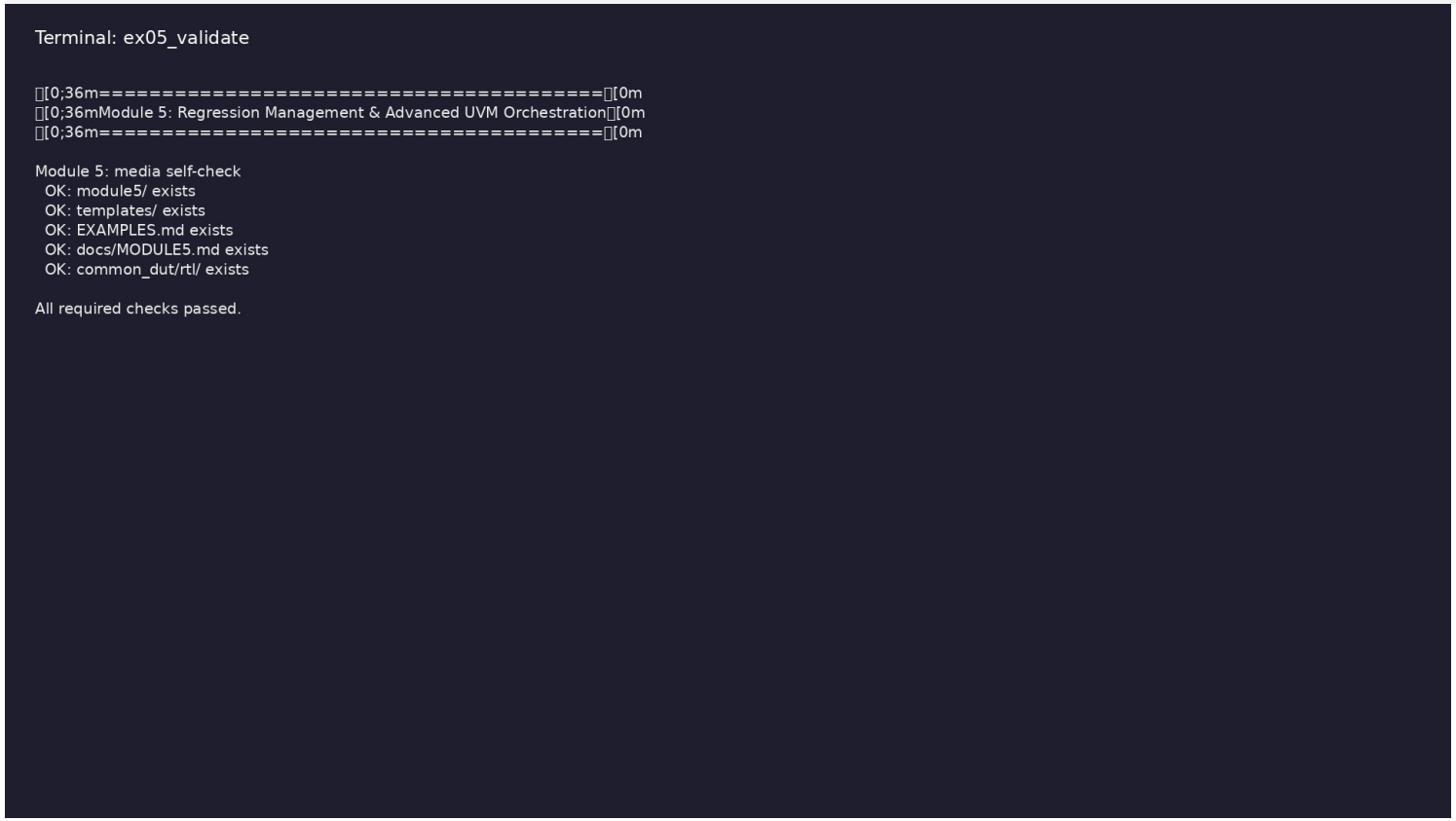
```
$ head -25 module5/templates/ADVANCED_UVM_PLAN.md
```

Demo: Reference regression ops

```
$ head -20 module5/.solutions/REGRESSION_OPS.md
```

Demo: Module validation

```
$ ./scripts/module5.sh --check
```



Practice & assessment

What you should know (1/5)

- Define and run repeatable regression flows across local and CI/farm environments.
- Use virtual sequences and advanced UVM patterns to orchestrate complex scenarios.
- Detect, triage, and manage flaky tests and instability in regressions.
- Implement a practical debug and observability strategy (logs, waves, metadata).
- Use coverage and bug data to prioritize and refine regression content.
- In `module5/REGRESSION_OPS.md`, define at least:

What you should know (2/5)

- A `sanity` job (fast, per-commit).
- A `core_nightly` job.
- A `stress_weekly` or `full_release` job.
- For each job, specify:
- Tier(s), test selection rules, expected runtime, and
  invocation commands.
- In `module5/ADVANCED_UVM_PLAN.md`, map several high-
  level tests to:

What you should know (3/5)

- Specific virtual sequences and their participating agents/sequencers.
- Key configuration parameters (modes, seeds, error-injection toggles).
- Identify at least 3 scenarios that require multi-agent coordination.
- In `module5/DEBUG_FLAKE_PLAN.md`, define:
- What constitutes a flake in your context.
- How many reruns you perform to confirm flakiness.

What you should know (4/5)

- How you quarantine and track flaky tests.
- How/when they must be fixed before sign-off.
- Define:
- Standard log naming and locations.
- When and how to generate waveforms.
- Minimum metadata required in failure reports (test ID, seed, tier, DUT hash).

What you should know (5/5)

- Verify on at least one failing test that you can reproduce and debug using these artifacts.

Exercises

- Runtime Tuning
- Coverage-Driven Job
- Resilience Drill

Assessment checklist

- Concrete regression jobs and commands captured in `module5/REGRESSION_OPS.md`.
- Virtual sequence and advanced UVM usage planned in `module5/ADVANCED_UVM_PLAN.md`.
- Flake and debug strategy documented in `module5/DEBUG_FLAKE_PLAN.md`.
- Regression flows demonstrably support coverage and sign-off goals.

Summary & next steps

- Turn your test, coverage, and environment designs into a **robust regression system** that scales:…
- Complete module5/CHECKLIST.md
- Review module5/EXAMPLES.md and run each lab
- Next: Next module in course